# Vocabulary Builder — Plan of Action

This plan assumes a small team, one deployable .NET API, one Angular application, and short one-week work packages. It favors incremental changes and preserves the existing architecture.

The target learning loop is:

**Discover → Save → Understand → Review → Practice → Measure → Resurface weak or overdue words**

---

# 1. Executive Plan of Action

Vocabulary Builder should evolve in three controlled stages:

1. **Secure and correct the existing application**
    - Fix secrets, passwords, authorization, quiz statistics, and `UserWord` identity.
    - Establish backend integration tests before structural refactoring.
2. **Create maintainable feature boundaries**
    - Standardize API contracts.
    - Separate dictionary lookup from personal vocabulary management.
    - Decompose the Angular vocabulary page.
    - Replace process-local quiz state.
3. **Build the learning loop**
    - Complete notes, examples, favorites, and archive behavior.
    - Add learning state, daily reviews, weak-word resurfacing, mastery, and progress.
    - Add advanced quiz modes.
    - Consider AI only after the core retention loop works.

A rewrite, microservices, NgRx, generalized repository framework, or large AI subsystem should not be part of the next 90 days.

---

# Part 1 — Fix Existing Weaknesses First

# 2. Immediate Critical Fixes

## R1. Rotate and externalize the JWT signing secret

- **Priority:** Critical

- **Size:** Small
- **Dependencies:** None

**Problem:**
A usable JWT signing key is committed in [appsettings.json (line 6)](F:/Source/VocabularyApp/VocabularyApp.WebApi/appsettings.json:6). A development key is also stored in [appsettings.Development.json (line 6)](F:/Source/VocabularyApp/VocabularyApp.WebApi/appsettings.Development.json:6).

**Why it matters:**
Security and production reliability. Anyone with the deployed signing key can create valid authentication tokens.

**Affected areas:**

- `VocabularyApp.WebApi`
- `appsettings.json`
- `appsettings.Development.json`
- `Program.cs`
- `JwtHelper`
- Deployment configuration
- All authenticated endpoints

**Recommended fix:**
Rotate every potentially deployed key. Keep only non-secret placeholders in tracked configuration. Load secrets from environment variables, user secrets, or the hosting provider's secret store. Fail startup when a production-grade key is absent.

**Step-by-step plan:**

1. Determine every environment in which the committed key may have been used.
2. Generate a new high-entropy key for each environment.
3. Configure those keys outside source control.
4. Change tracked configuration to contain no functional production secret.
5. Verify `Program.cs` and `JwtHelper` consume the same configuration entry.
6. Redeploy the API, invalidating existing tokens.
7. Document local secret setup without recording actual values.
8. Scan tracked history and deployment configuration for other credentials.

**Testing required:**

- API starts with a valid externally supplied key.
- API fails clearly when the key is missing.
- Tokens signed with the old key fail.
- New login tokens access protected endpoints.

- Tokens with incorrect issuer, audience, signature, or expiry fail.
- Manually verify secrets do not appear in build artifacts or logs.

**Definition of done:**

- No operational signing key is tracked.
- All deployed keys are rotated.
- Production startup requires externally supplied secure configuration.
- Authentication regression tests pass.
- Secret setup is documented.

---

# R2. Replace SHA-256 password hashing safely

- **Priority:** Critical
- **Size:** Medium
- **Dependencies:** R1 recommended but not technically required

**Problem:**

[PasswordHelper.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Helpers/PasswordHelper.cs) uses salted SHA-256. SHA-256 is deliberately fast and unsuitable for password storage.

**Why it matters:**
Security. A database leak would allow efficient offline password guessing.

**Affected areas:**

- `PasswordHelper`
- `UserService`
- `User.PasswordHash`
- Register, login, and change-password endpoints
- Existing user records

**Recommended fix:**
Use ASP.NET Core `PasswordHasher<User>` or another adaptive password hash. Support transparent migration: verify the legacy format once, then replace it with the new hash after a successful login.

**Step-by-step plan:**

1. Inventory the current stored format: `salt:hash`.
2. Introduce an injectable password-hashing abstraction.
3. Implement the modern hasher.
4. Retain a narrowly scoped legacy verifier.
5. During login:

    - Recognize the stored format.
    - Verify it.
    - Rehash immediately when legacy or when rehash is recommended.
6. Use only the new format for registration and password changes.
7. Avoid changing the database column unless the modern format exceeds its current length.
8. Add telemetry for successful migrations without logging hashes or passwords.
9. Remove the legacy verifier only after the migration window is complete.

**Testing required:**

- Existing legacy user can log in.
- Successful legacy login updates the hash.
- Wrong legacy password does not migrate anything.
- New registration creates a modern hash.
- Password change creates a modern hash.
- Malformed hashes fail safely.
- Verify passwords and hashes never enter logs.

**Definition of done:**

- New passwords use an adaptive algorithm.
- Existing users migrate without forced resets.
- No plaintext password or hash is logged.
- Authentication tests cover both formats.
- A documented date or condition exists for legacy-code removal.

---

# R3. Secure or remove the public canonical word-write endpoint

- **Priority:** Critical
- **Size:** Small
- **Dependencies:** Product decision about administration

**Problem:**

`POST /api/words/add` is described as an administrative operation but is publicly accessible in [WordsController.cs (line 58)](F:/Source/VocabularyApp/VocabularyApp.WebApi/Controllers/WordsController.cs:58). It also returns success even when the service fails.

**Why it matters:**

Security and data correctness. An unauthenticated caller can pollute the canonical dictionary.

**Affected areas:**

- `WordsController.AddWord`
- `WordService.AddWordAsync`
- `Word`, `WordDefinition`, and `PartOfSpeech`
- API documentation

**Recommended fix:**

Remove the endpoint if it has no current user workflow. Otherwise require an explicit administrator policy and return accurate status codes.

**Step-by-step plan:**

1. Search clients and scripts for legitimate usage.
2. Decide whether canonical manual entry is currently needed.
3. If not needed, remove the route and unused service operation.
4. If needed:
    - Add a real role/claim model.
    - Add an authorization policy.
    - Validate the request.
    - Return conflict or validation responses correctly.
5. Remove commented-out result checking.
6. Update Swagger and HTTP examples.

**Testing required:**

- Anonymous request receives 401 or route-not-found.
- Ordinary authenticated user receives 403 if the endpoint remains.
- Authorized administrator can add valid data.
- Service failure does not return success.
- Duplicate canonical entries are handled deterministically.

**Definition of done:**

- No anonymous canonical mutation is possible.
- Authorization behavior is covered by integration tests.
- Endpoint responses accurately represent results.
- Documentation matches the decision.

---

# R4. Correct quiz counters and review timestamps

- **Priority:** Critical
- **Size:** Medium
- **Dependencies:** Initial backend test harness

**Problem:**

[QuizService.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Services/QuizService.cs)

stores `QuizResult` rows but does not update `UserWord.CorrectAnswers`, `TotalAttempts`, `LastReviewedAt`, or `LastCorrectAt`. Per-word accuracy exposed by `UserVocabularyItemDto` is therefore unreliable.

**Why it matters:**
Data correctness and user trust. Weak-word, mastery, and progress features cannot use inaccurate data.

**Affected areas:**

- `QuizService.SubmitQuizAsync`
- `UserWord`
- `QuizResult`
- `UserVocabularyItemDto.AccuracyRate`
- Quiz history and future review scheduling

**Recommended fix:**
Persist attempt history and update aggregate learning state in one database transaction. Count unanswered questions according to an explicit product rule.

**Step-by-step plan:**

1. Decide whether unanswered submitted questions count as attempts and incorrect answers.
2. Define the meaning of each existing timestamp and counter.
3. Load the affected `UserWord` rows for the authenticated user.
4. Validate submitted question and option identifiers.
5. Create immutable `QuizResult` records.
6. Increment attempts and correct counts.
7. Set `LastReviewedAt`; set `LastCorrectAt` only for correct answers.
8. Save all changes transactionally.
9. Prevent duplicate submission of the same quiz session.
10. Decide whether old result rows should backfill existing counters.
11. If backfilling, create an idempotent migration or maintenance operation and verify totals first.

**Testing required:**

- Correct and incorrect answers update appropriate fields.
- Unanswered question follows the chosen rule.
- Mixed results update multiple words correctly.
- Invalid ownership changes no records.
- Duplicate submission does not double-count.
- Transaction rollback leaves both attempts and counters unchanged after failure.
- Historical backfill is idempotent if performed.

**Definition of done:**

- Per-word totals agree with persisted quiz results.
- Submission is atomic and idempotent.
- Product rules for unanswered questions are documented.
- Integration tests cover correctness and rollback.

---

# R5. Correct `UserWord` identity to one saved word per user

- **Priority:** Critical
- **Size:** Large
- **Dependencies:** R4; explicit product confirmation

**Problem:**
The current unique index is `(UserId, WordId, PartOfSpeechId)`, allowing the same canonical word to appear multiple times for one user. The desired model is one saved word per user, with the chosen meaning represented by `PreferredWordDefinitionId`.

**Why it matters:**
Data correctness, UX, and future review scheduling. Multiple rows for one saved word make mastery, favorites, notes, and due dates ambiguous.

**Affected areas:**

- `UserWord`
- `ApplicationDbContext`
- EF migrations
- `WordService.AddToVocabularyAsync`
- `SetPreferredDefinitionAsync`
- `QuizService`
- Vocabulary DTOs and Angular models
- Existing duplicate data

**Recommended fix:**
Change uniqueness to `(UserId, WordId)`. Treat `PreferredWordDefinitionId` as the selected meaning. Remove `PartOfSpeechId` from `UserWord` if it can always be derived from the preferred definition.

**Step-by-step plan:**

1. Confirm the product rule: one saved word may have one selected meaning at a time.
2. Query existing data for duplicate `(UserId, WordId)` groups.
3. Define a deterministic merge policy:
    - Keep one row.

    - Preserve favorite if any duplicate is favorite.
    - Merge or select notes deliberately.
    - Sum counters only after checking for duplicate history.
    - Select the explicit preferred definition when available.
    - Reassign dependent `QuizResult` and `SampleSentence` rows.
4. Add a data-cleanup migration before adding the new unique constraint.
5. Add a unique index on `(UserId, WordId)`.
6. Decide whether to remove `PartOfSpeechId`:
    - If removed, migrate preferred-definition relationships first.
    - If retained temporarily, stop using it as identity.
7. Change add behavior to return a conflict or existing record instead of creating another sense.
8. Simplify preferred-definition updates so they do not move an entry between part-of-speech identities.
9. Update quiz and vocabulary projections.
10. Update Angular assumptions and models.
11. Validate the migration against a production-like database copy.

**Testing required:**

- Duplicate migration preserves dependent data.
- A user cannot save the same word twice.
- Different users can save the same word.
- Preferred definition must belong to the saved word.
- Changing preferred definition preserves notes, favorite state, and history.
- Rollback and backup/restore procedure tested manually.

**Definition of done:**

- Database enforces one `UserWord` per user and canonical word.
- No orphaned attempts or examples exist.
- API and UI no longer treat part of speech as saved-entry identity.
- Migration is documented and tested with duplicate sample data.

---

# 3. Remediation Backlog

## R6. Add a backend integration-test foundation

- **Priority:** High
- **Size:** Large
- **Dependencies:** None; begin before major refactoring

**Problem:**
There is no backend test project, leaving authentication, ownership, EF relationships, and quiz behavior unprotected.

**Why it matters:**
Maintainability, security, and refactoring safety.

**Affected areas:**
Entire Web API and data layer, especially `UsersController`, `WordsController`, `QuizController`, and their services.

**Recommended fix:**
Create an API integration-test project using `WebApplicationFactory`. Use a relational test database compatible with production behavior; avoid relying exclusively on EF's in-memory provider.

**Steps:**

1. Make the application host discoverable by the test project.
2. Add isolated database creation and teardown.
3. Add user/token test helpers.
4. Cover authentication and authorization first.
5. Add ownership, vocabulary, lookup, and quiz scenarios.
6. Run tests in CI.
7. Add focused unit tests only for pure rules.

**Testing required:**
The test harness itself must demonstrate isolation, deterministic seeding, and parallel safety.

**Definition of done:**
Critical API journeys run automatically and reliably from a clean checkout.

---

# R7. Standardize API success and error contracts

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R6

**Problem:**
The API mixes anonymous envelopes, `ApiResponse`, multiple `ApiResult` types, `ServiceResult<T>`, `Error`, `ErrorMessage`, and `Message`.

**Why it matters:**
Maintainability and frontend reliability.

**Affected areas:**

- All controllers and DTOs
- `DTOs/ApiResponse.cs`
- `DTOs/ApiResult.cs`
- controller-local `ApiResult<T>`
- `ServiceResult<T>`
- Angular `ApiResponse<T>`
- Component error handling

**Recommended fix:**
Use typed successful response DTOs and standard `ProblemDetails` errors, or one consistent typed envelope. Prefer `ProblemDetails` because HTTP status already communicates result class.

**Steps:**

1. Inventory current frontend-consumed shapes.
2. Choose the target contract.
3. Define stable validation, not-found, conflict, unauthorized, and server-error forms.
4. Migrate one vertical slice, preferably vocabulary.
5. Update Angular API clients.
6. Migrate authentication and quiz.
7. Remove duplicate result classes.
8. Update Swagger and tests.

**Testing required:**
Contract tests for every status category; Angular tests for centralized error parsing.

**Definition of done:**
Every endpoint follows one documented convention, and components no longer inspect several possible error properties.

---

# R8. Centralize exception handling and request validation

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R7

**Problem:**
Controllers repeat broad `try/catch` blocks, claim parsing, model-state handling, logging, and anonymous 500 responses.

**Why it matters:**
Security, maintainability, and reliable status codes.

**Affected areas:**
`Program.cs`, all controllers, request DTOs, logging.

**Recommended fix:**
Add centralized exception handling, standard validation responses, and a current-user accessor. Keep expected domain outcomes out of exceptions.

**Steps:**

1. Add global exception middleware/handler.
2. Return standardized problem details with correlation ID.
3. Add data annotations or focused validators to request DTOs.
4. Centralize authenticated user ID retrieval.
5. Remove repetitive controller catches incrementally.
6. Map domain outcomes to 400/404/409 consistently.
7. Prevent internal exception messages from reaching clients.

**Testing required:**
Malformed request, missing claim, domain conflict, unknown resource, and unexpected exception tests.

**Definition of done:**
Controllers primarily coordinate requests; unexpected errors are logged once and returned consistently.

---

# R9. Clean entities, DTOs, and placeholder fields

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R5 decision; R6

**Problem:**

- `WordDefinition` contains `[NotMapped] Success` and `Data`.
- `UserWord` has duplicate `CreatedAt`/`AddedAt`.
- `CustomDefinition` and `DifficultyLevel` are exposed but not persisted.
- Comments contradict actual part-of-speech relationships.
- `ChatHistory` and `SampleSentence` imply incomplete features.
- DTOs advertise fields the application cannot reliably save.

**Why it matters:**
Data correctness and maintainability.

**Affected areas:**
Data models, `ApplicationDbContext`, migrations, vocabulary DTOs, Angular models.

**Recommended fix:**
Remove response concerns from entities, remove or deliberately implement placeholders, and make API contracts reflect real capabilities.

**Steps:**

1. Classify each property as retained, implemented, deferred, or removed.
2. Remove non-database response fields from `WordDefinition`.
3. Remove duplicate timestamps.
4. Remove non-persisted DTO fields unless scheduled for immediate implementation.
5. Align comments and nullability.
6. Generate a minimal migration only for actual schema changes.
7. Update mappings and Angular interfaces.
8. Verify older JSON clients if compatibility matters.

**Testing required:**
Migration test, serialization tests, vocabulary CRUD regression, and schema snapshot review.

**Definition of done:**
Every exposed field has a clear persistence and business meaning.

---

# R10. Extract the external dictionary provider from `WordService`

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R6–R8

**Problem:**
`WordService` directly builds provider URLs, sends HTTP requests, maps provider DTOs, resolves parts of speech, and persists cache data.

**Why it matters:**
Maintainability and production resilience.

**Affected areas:**
`WordService`, external DTOs, `HttpClient` registration, `PartOfSpeech`, tests.

**Recommended fix:**
Introduce an `IDictionaryProvider` returning a provider-neutral dictionary result. Keep persistence in a separate canonical dictionary service.

**Steps:**

1. Define a provider-neutral result model.
2. Move URL construction and external DTO mapping into a typed client.
3. Configure base URL and timeout.
4. Add cancellation tokens.
5. Define provider-not-found versus provider-unavailable outcomes.
6. Add mapping behavior for unknown parts of speech; do not silently force noun.
7. Add mock-provider unit tests and HTTP contract tests.
8. Move callers without changing external API behavior.

**Testing required:**
Success, 404, timeout, malformed response, no audio, multiple meanings, and unknown part-of-speech cases.

**Definition of done:**
`WordService` no longer knows dictionaryapi.dev's transport format.

---

# R11. Extract personal vocabulary responsibilities from `WordService`

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R5, R7–R10

**Problem:**
`WordService` combines canonical lookup with personal vocabulary commands, search, favorites, and preferred definitions.

**Why it matters:**
Maintainability and feature growth.

**Affected areas:**
`WordService`, `IWordService`, `WordsController`, vocabulary DTOs.

**Recommended fix:**
Create a focused `VocabularyService`. Keep canonical lookup under a dictionary service. A controller split is optional but recommended once routes are stable.

**Steps:**

1. Identify dictionary versus vocabulary operations.
2. Move vocabulary queries with unchanged behavior.
3. Move add/favorite/preferred-definition commands.

4. Centralize vocabulary DTO projection.
5. Add notes/archive operations only after the split.
6. Rename interfaces to reflect responsibility.
7. Remove `ServiceResult<object>` in favor of typed results.

**Testing required:**
All existing vocabulary integration tests must pass unchanged during extraction.

**Definition of done:**
Dictionary provider, canonical dictionary, and user vocabulary each have a clear responsibility.

---

# R12. Replace static in-memory quiz sessions

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R4, R6–R8

**Problem:**
Quiz sessions live in a static `ConcurrentDictionary`. They disappear during restart and are not shared across API instances.

**Why it matters:**
Production reliability and duplicate-submission control.

**Affected areas:**
`QuizService`, quiz DTOs, future `QuizSession` entity, cleanup behavior.

**Recommended fix:**
Persist quiz sessions and question state. For a small application, SQL persistence is simpler than introducing Redis.

**Steps:**

1. Define quiz-session lifetime and refresh/resume behavior.
2. Add `QuizSession` and `QuizQuestion` persistence, or a compact serialized question-state field if carefully versioned.
3. Store owner, mode, creation, expiry, and completion time.
4. Store correct-answer state only server-side.
5. Submit and mark completed transactionally.
6. Reject expired, foreign, or completed sessions.
7. Add a scheduled or request-triggered cleanup strategy.
8. Remove the static dictionary.

**Testing required:**
Restart/resume, expiry, ownership, duplicate submission, concurrent submission, and cleanup tests.

**Definition of done:**
An active quiz survives process restart and cannot be scored twice.

---

# R13. Handle concurrent dictionary cache misses safely

- **Priority:** High
- **Size:** Medium
- **Dependencies:** R10; unique canonical-word policy

**Problem:**
Two requests can miss the same word, both call the provider, and both try to insert a row protected by the unique `Word.Text` index.

**Why it matters:**
Data correctness and intermittent production reliability.

**Affected areas:**
Dictionary lookup/persistence service, `Word`, `WordDefinition`, EF transactions.

**Recommended fix:**
Normalize lookup keys, preserve database uniqueness as the final guard, and handle duplicate-insert races by re-reading the winning row.

**Steps:**

1. Define word normalization, including trim and case.
2. Query by the normalized key.
3. Fetch provider data.
4. Insert word and definitions in one transaction.
5. On recognized unique-key conflict, clear failed tracked state and reload the existing word.
6. Avoid inserting partial canonical words.
7. Consider short-lived per-process request coalescing only as an optimization.

**Testing required:**
Parallel lookup integration test, failure between word and definitions, case variants, and provider timeout.

**Definition of done:**
Concurrent identical lookups return one canonical record without user-visible 500 responses.

---

## R14. Decompose `WordLookupComponent`

- **Priority:** High
- **Size:** Large
- **Dependencies:** R7 and stable vocabulary APIs

**Problem:**
The 780-line component and 443-line template manage lookup, suggestions, collection browsing, favorites, modal editing, audio, highlights, and toast rendering.

**Why it matters:**
Maintainability, accessibility, and regression risk.

**Affected areas:**
`WordLookupComponent`, its template/styles/spec, route structure.

**Recommended fix:**
Split by visible responsibility while keeping one feature route initially.

**Steps:**

1. Add characterization tests for existing behavior.
2. Extract the application-level toast host.
3. Extract stateless definition rendering.
4. Extract preferred-definition dialog.
5. Extract vocabulary list/card/filter components.
6. Extract word search and suggestions.
7. Move HTTP calls into feature API services.
8. Keep page-level orchestration in a small container.
9. Replace duplicated markup and `any` types.
10. Evaluate separate `/discover` and `/words` routes after behavior stabilizes.

**Testing required:**
Component tests for child inputs/outputs and an end-to-end search/save/favorite/edit flow.

**Definition of done:**
The route-level component coordinates feature state but does not render or implement every subfeature itself.

---

## R15. Improve Angular API and authentication handling

- **Priority:** High

- **Size:** Medium
- **Dependencies:** R7; coordinate with R14

**Problem:**
Components construct endpoint strings, error handling is duplicated, authentication headers are manually attached, and tokens are stored in `localStorage`.

**Why it matters:**
Security and maintainability.

**Affected areas:**
`ApiService`, `AuthService`, guard, Angular configuration, all page components.

**Recommended fix:**
Introduce an authentication interceptor and typed feature API services. Decide whether secure HttpOnly cookies are viable for the deployment topology.

**Steps:**

1. Create typed `DictionaryApi`, `VocabularyApi`, and `QuizApi` services.
2. Add bearer-token interceptor as the short-term improvement.
3. Centralize 401 handling and logout behavior.
4. Return a `UrlTree` from the route guard.
5. Remove `any` response types.
6. Evaluate cookie-based authentication, CSRF requirements, and same-site deployment.
7. If cookies are adopted, migrate in a separate tested change.
8. Align registration behavior with the returned authentication response.

**Testing required:**
Interceptor tests, 401 behavior, expired token behavior, login/logout, guard redirects, and registration journey.

**Definition of done:**
Components do not manage authorization headers or generic error-shape parsing.

---

# R16. Restore server-side paging and search

- **Priority:** Medium
- **Size:** Medium
- **Dependencies:** R11 and typed Angular API clients

**Problem:**
The UI requests up to 1,000 vocabulary entries and performs filtering and alphabetical browsing locally despite existing backend query support.

**Why it matters:**
Performance and production reliability as collections grow.

**Affected areas:**
`GetUserVocabularyAsync`, `SearchUserVocabularyAsync`, `WordsController`, vocabulary UI.

**Recommended fix:**
Use bounded server-side paging, searching, letter filtering, favorite filtering, and later learning-state filtering.

**Steps:**

1. Define one query contract: page, page size, term, initial letter, favorite, archived.
2. Set a reasonable maximum page size.
3. Add deterministic ordering.
4. Add database indexes after examining generated queries.
5. Return total counts and active filters.
6. Debounce frontend search and cancel stale requests.
7. Preserve filters in URL query parameters if useful.
8. Remove the 1,000-record workaround.

**Testing required:**
Paging boundaries, combined filters, empty results, case behavior, rapid search changes, and large seeded collection.

**Definition of done:**
Frontend never loads an unbounded collection for ordinary browsing.

---

# R17. Fix accessibility, dead controls, UI inconsistencies, and documentation

- **Priority:** Medium
- **Size:** Medium
- **Dependencies:** Prefer after R14, except misleading controls can be fixed immediately

**Problem:**

- "Save for Later" has no behavior.
- Dashboard cards are clickable `<div>` elements.
- Autocomplete lacks complete keyboard/ARIA semantics.
- Toasts are scoped to one page.
- Several emoji strings appear encoding-damaged.
- Dashboard concepts and documentation are stale.

- Angular build reports an SCSS budget warning.
- Angular tests do not complete reliably.

**Why it matters:**
User experience, accessibility, and developer confidence.

**Affected areas:**
Dashboard, vocabulary templates, toast host, styles, README, HTTP examples, test configuration.

**Recommended fix:**
Remove misleading affordances, use semantic controls, establish consistent navigation and feedback, and refresh developer documentation.

**Steps:**

1. Remove or implement “Save for Later.”
2. Convert dashboard cards to links/buttons.
3. Add accessible labels, focus behavior, combobox/listbox semantics, and dialog focus management.
4. Move toast rendering to the application shell with live-region behavior.
5. Verify UTF-8 and replace unstable emoji with consistent icons or text.
6. Fix the SCSS budget or adjust it deliberately with justification.
7. diagnose the test-runner timeout.
8. Update README, frontend summary, routes, and HTTP examples.
9. Perform keyboard-only and screen-reader smoke testing.

**Testing required:**
Automated accessibility scan, keyboard walkthrough, build, unit suite, and manual mobile checks.

**Definition of done:**
No visible dead action remains, primary flows are keyboard-usable, tests terminate reliably, and documentation matches the application.

---

# 4. Immediate Work — Do These First

Exact recommended order:

1. **R1 — Rotate and externalize JWT secrets**
2. **R3 — Close the public canonical write endpoint**
3. **R2 — Introduce modern password hashing with legacy migration**
4. **R6 — Establish backend integration tests**

5. **R4 — Fix quiz attempt aggregates and transactional submission**
6. **R5 — Migrate to one saved word per user**
7. **R7 — Standardize API contracts**
8. **R8 — Centralize exception handling and validation**

R1 and R3 are intentionally ahead of test infrastructure because they are narrowly scoped security exposures. Major data and architectural changes begin only after the test harness exists.

---

# 5. Sprint-by-Sprint Cleanup Plan

## Sprint 1 — Close immediate security exposures

**Goal:** Remove known token, password, and unauthorized-write risks.

**Tasks:** R1, R3, begin R2.

**Dependencies:** Access to deployment configuration and product decision about administration.

**Expected result:** New signing keys, no public canonical mutation, modern hashes for new or changed passwords.

**Test before moving on:**

- Old tokens rejected.
- Login and protected endpoints work with new configuration.
- Anonymous/ordinary users cannot mutate canonical data.
- New and legacy password flows pass.

## Sprint 2 — Establish the safety net and correct quiz data

**Goal:** Protect critical behavior and make current statistics trustworthy.

**Tasks:** R6 foundation, R4.

**Dependencies:** Stable test database strategy.

**Expected result:** Automated auth/ownership/quiz tests and atomic counter updates.

**Test before moving on:**

- Full critical integration suite.
- Duplicate quiz submission.
- Transaction rollback.
- Cross-user access denial.

# Sprint 3 — Correct saved-word identity and schema debt

**Goal:** Establish one unambiguous learning record per user and word.

**Tasks:** R5, R9.

**Dependencies:** Database backup, duplicate-data inventory, product confirmation.

**Expected result:** One `UserWord` per user/word and clean entity/DTO semantics.

**Test before moving on:**

- Migration using duplicate production-like data.
- Dependent quiz/example preservation.
- Preferred-definition changes.
- Rollback procedure.

# Sprint 4 — Stabilize backend contracts

**Goal:** Make API behavior predictable before service extraction.

**Tasks:** R7, R8.

**Dependencies:** Integration suite and updated frontend coordination.

**Expected result:** Consistent errors, typed responses, centralized exception handling.

**Test before moving on:**

- Contract tests for all response categories.
- Angular login, vocabulary, and quiz regression.
- No internal exception leakage.

# Sprint 5 — Separate dictionary and vocabulary boundaries

**Goal:** Reduce `WordService` coupling.

**Tasks:** R10, R11, R13.

**Dependencies:** Stable contracts and characterization tests.

**Expected result:** Provider-neutral external lookup, focused vocabulary service, race-safe caching.

**Test before moving on:**

- Cache hit/miss/provider failure.

- Concurrent identical lookup.
- Vocabulary commands and queries.
- Unknown part-of-speech behavior.

# Sprint 6 — Make quiz sessions production-safe

**Goal:** Support restart-safe and idempotent quizzes.

**Tasks:** R12.

**Dependencies:** Correct quiz submission and stable schema migration process.

**Expected result:** Persisted quiz sessions and no static state.

**Test before moving on:**

- Restart/resume.
- Expiry and cleanup.
- Concurrent/duplicate submission.
- Cross-user session access.

# Sprint 7 — Restructure Angular feature boundaries

**Goal:** Make the UI safe to extend.

**Tasks:** R14, R15.

**Dependencies:** Stable typed backend contracts.

**Expected result:** Focused feature components, typed API clients, centralized authentication behavior.

**Test before moving on:**

- Search/save/favorite/preferred-definition end-to-end.
- Interceptor and guard tests.
- Component interaction tests.
- Responsive regression.

# Sprint 8 — Scale and polish the current experience

**Goal:** Complete existing workflows before adding learning features.

**Tasks:** R16, R17, archive/delete foundation.

**Dependencies:** Vocabulary service and component split.

**Expected result:** Server-side querying, accessible navigation, no dead controls, reliable tests and documentation.

**Test before moving on:**

- Large vocabulary collection.
- Keyboard-only workflow.
- Automated accessibility scan.
- Clean production builds and terminating test runs.

---

# Part 2 — Feature and Product Enhancement Roadmap

# 6. Feature Enhancement Backlog

| Order | Feature | Priority | Timing | Size | Risk |
|---|---|---|---|---|---|
| 1 | Archive/delete saved words | Must Have | Next | Small | Low |
| 2 | Favorites filtering | Must Have | Next | Small | Low |
| 3 | Personal notes | Must Have | Next | Medium | Low |
| 4 | Personal example sentences | High Value | Next | Medium | Medium |
| 5 | Review history | Must Have | Soon | Medium | Medium |
| 6 | Learning state and mastery levels | Must Have | Soon | Medium | Medium |
| 7 | Daily review queue | Must Have | Soon | Large | Medium |
| 8 | Weak-word review | Must Have | Soon | Medium | Medium |
| 9 | Simple spaced repetition | Must Have | Soon | Large | High |
| 10 | Progress dashboard | High Value | Soon | Medium | Medium |
| 11 | Typed recall | High Value | Later | Medium | Medium |
| 12 | Multiple quiz modes | High Value | Later | Medium | Medium |

| | | | | | |
|---|---|---|---|---|---|
| 13 | Usage/context quizzes | High Value | Later | Large | High |
| 14 | Daily learning goals | High Value | Later | Medium | Medium |
| 15 | Streaks | Useful Later | Later | Medium | Medium |
| 16 | Word collections/tags | Useful Later | Later | Medium | Medium |
| 17 | Import/export | Useful Later | Later | Medium | Medium |
| 18 | AI explanations | Future / Experimental | Much Later | Medium | High |
| 19 | AI-generated examples | Future / Experimental | Much Later | Medium | High |
| 20 | AI vocabulary coach | Future / Experimental | Much Later | Large | High |
| 21 | Conversational practice | Future / Experimental | Much Later | Large | High |

---

# 7. Feature Implementation Plans

## F1. Archive/delete saved words

- **Benefit:** Correct mistakes and remove irrelevant vocabulary.
- **Priority/Timing:** Must Have / Next
- **Size/Risk:** Small / Low
- **Prerequisites:** R5, R7, R11
- **Data model:** Prefer `ArchivedAt` for recoverability; hard delete can follow explicit confirmation.
- **Backend:** Ownership-protected archive, restore, and optional delete endpoints.
- **Frontend:** Actions on word detail/card, confirmation, archived filter.
- **Testing:** Cross-user denial, archive exclusion, restore, dependent history behavior.
- **Steps:** Define archive semantics → add field/migration → add API → add UI → add cleanup policy.
- **Done:** User can remove a word from active learning without corrupting history.

## F2. Favorites filtering

- **Benefit:** Makes the existing favorite flag useful for prioritization.
- **Priority/Timing:** Must Have / Next

- **Size/Risk:** Small / Low
- **Prerequisites:** R16
- **Data model:** None.
- **Backend:** Add `favorite` vocabulary query filter.
- **Frontend:** Filter chip and clear active state.
- **Testing:** Combined search/letter/favorite paging.
- **Steps:** Extend query contract → add test → implement filter → add UI.
- **Done:** Favorites can be listed predictably without loading all words.

# F3. Personal notes

- **Benefit:** Lets users record mnemonics, translations, and distinctions meaningful to them.
- **Priority/Timing:** Must Have / Next
- **Size/Risk:** Medium / Low
- **Prerequisites:** R5, R9, R11
- **Data model:** Retain `UserWord.PersonalNotes`; confirm maximum length.
- **Backend:** Typed patch/update command with ownership and length validation.
- **Frontend:** Notes editor on saved-word detail with save/cancel feedback.
- **Testing:** Ownership, limits, blank/null behavior, persistence.
- **Steps:** Define UX → finalize DTO → add API → add editor → add tests.
- **Done:** A note can be safely created, edited, cleared, and retrieved.

# F4. Personal example sentences

- **Benefit:** Moves a word from recognition to personally meaningful context.
- **Priority/Timing:** High Value / Next
- **Size/Risk:** Medium / Medium
- **Prerequisites:** R5, `SampleSentence` cleanup, F3 patterns
- **Data model:** Simplify `SampleSentence` ownership through `UserWord`; optionally add `UpdatedAt`.
- **Backend:** User-example CRUD, ownership validation, limits.
- **Frontend:** Example list/editor distinct from dictionary-provider examples.
- **Testing:** Ownership, validation, archive behavior, multiple examples.
- **Steps:** Clarify provider versus personal example labels → clean entity → migrate → APIs → UI.
- **Done:** Users can manage personal examples without confusing them with canonical content.

# F5. Review history

- **Benefit:** Shows what was practiced and creates trustworthy feedback.
- **Priority/Timing:** Must Have / Soon
- **Size/Risk:** Medium / Medium

- **Prerequisites:** R4, R12
- **Data model:** Persisted `QuizSession` with completion and mode; retain per-question results.
- **Backend:** Paginated history and session-detail endpoints.
- **Frontend:** Review-history page or expandable recent activity.
- **Testing:** Session grouping, paging, scores, historical ownership.
- **Steps:** Stabilize session model → query DTOs → APIs → list/detail UI.
- **Done:** User can inspect completed sessions and question outcomes.

# F6. Learning state and mastery levels

- **Benefit:** Communicates whether a word is new, learning, familiar, or mastered.
- **Priority/Timing:** Must Have / Soon
- **Size/Risk:** Medium / Medium
- **Prerequisites:** R4, R5, F5
- **Data model:** Add mastery state and scheduling fields, preferably in a one-to-one `ReviewState`.
- **Backend:** Deterministic transition policy driven by review results.
- **Frontend:** Mastery label and explanation; avoid manual mastery initially.
- **Testing:** State transitions, lapses, boundary cases, backfill defaults.
- **Steps:** Define states → define transitions → add schema → apply transactionally → expose read model.
- **Done:** Every active word has an explainable learning state based on actual practice.

# F7. Daily review queue

- **Benefit:** Gives the user one clear daily action.
- **Priority/Timing:** Must Have / Soon
- **Size/Risk:** Large / Medium
- **Prerequisites:** F6, persisted sessions, server-side filtering
- **Data model:** `NextReviewAt`, active/archive state, scheduling metadata.
- **Backend:** Endpoint selecting due words with stable ordering and daily limit.
- **Frontend:** “Today” home, due count, start-review flow, empty state.
- **Testing:** Due boundaries, timezone, archived words, queue stability, concurrent sessions.
- **Steps:** Decide timezone/day semantics → query policy → endpoint → dashboard → review launch.
- **Done:** User can complete a bounded daily queue and immediately see remaining due work.

# F8. Weak-word review

- **Benefit:** Concentrates effort on frequently missed vocabulary.
- **Priority/Timing:** Must Have / Soon

- **Size/Risk:** Medium / Medium
- **Prerequisites:** R4, F6
- **Data model:** May derive weakness first; optionally add `LapseCount`.
- **Backend:** Define weakness using recent performance, not only lifetime accuracy.
- **Frontend:** Weak-word filter and focused review option with explanation.
- **Testing:** New words, sparse history, recent recovery, ties.
- **Steps:** Define formula → implement pure tested policy → add query → add UI.
- **Done:** Weak-word selection is deterministic and traceable to recent attempts.

# F9. Simple spaced repetition

- **Benefit:** Reviews words near the point of forgetting.
- **Priority/Timing:** Must Have / Soon
- **Size/Risk:** Large / High
- **Prerequisites:** F6–F8 and reliable timestamps
- **Data model:** Interval, ease or stage, next review, consecutive correct, lapse count.
- **Backend:** One documented scheduling policy invoked transactionally after review.
- **Frontend:** Due status and simple feedback choices if the algorithm needs them.
- **Testing:** Extensive deterministic scheduling tests, timezone/DST, repeated failure, long intervals.
- **Steps:** Select a simple policy → simulate it → define migration defaults → implement as a pure service → integrate → monitor.
- **Done:** Scheduling is deterministic, explainable, tested, and can be revised without rewriting quiz logic.

# F10. Progress dashboard

- **Benefit:** Shows useful outcomes: reviews completed, due words, weak words, and mastery movement.
- **Priority/Timing:** High Value / Soon
- **Size/Risk:** Medium / Medium
- **Prerequisites:** F5–F9
- **Data model:** Prefer queries over duplicate summary tables initially.
- **Backend:** Aggregated endpoints with bounded time windows.
- **Frontend:** Replace placeholder dashboard cards with actionable metrics.
- **Testing:** Empty/new user, date boundaries, aggregation correctness.
- **Steps:** Select 4–6 actionable metrics → validate queries → build cards/trends → link each metric to action.
- **Done:** Every displayed metric is accurate and guides a next step.

# F11. Typed recall

- **Benefit:** Tests active memory rather than recognizing an option.

- **Priority/Timing:** High Value / Later
- **Size/Risk:** Medium / Medium
- **Prerequisites:** Extensible quiz generator and scoring boundaries
- **Data model:** Existing answer fields can be used; add scoring metadata if needed.
- **Backend:** Normalize case/spacing; define exact versus tolerant matching.
- **Frontend:** Text-answer question, feedback, accessibility.
- **Testing:** Capitalization, whitespace, punctuation, variants, empty answers.
- **Steps:** Define accepted-answer policy → add question type → server scoring → UI → compare outcomes.
- **Done:** Typed answers are scored consistently and explained transparently.

# F12. Multiple quiz modes

- **Benefit:** Lets users practice recognition, recall, and meaning selection.
- **Priority/Timing:** High Value / Later
- **Size/Risk:** Medium / Medium
- **Prerequisites:** Quiz generation extracted from persistence/scoring
- **Data model:** Store real mode/question type in session and result.
- **Backend:** Strategy-based question generators without a large framework.
- **Frontend:** Plain-language mode choices and sensible defaults.
- **Testing:** Each generator, mixed mode, insufficient vocabulary, scoring.
- **Steps:** Extract current generator → add modes individually → store type → expose selection.
- **Done:** Modes generate valid questions and history reports the actual type.

# F13. Usage/context quizzes

- **Benefit:** Tests whether the learner understands how a word works in a sentence.
- **Priority/Timing:** High Value / Later
- **Size/Risk:** Large / High
- **Prerequisites:** Personal/canonical examples, multiple quiz modes
- **Data model:** Example source and answer metadata.
- **Backend:** Cloze generation and validation; initially use curated/provider examples.
- **Frontend:** Sentence prompt and contextual feedback.
- **Testing:** Ambiguous blanks, multiple valid answers, missing examples.
- **Steps:** Establish example quality rules → build deterministic cloze mode → pilot → consider AI generation later.
- **Done:** Context questions have one defensible expected answer and useful feedback.

# F14. Daily learning goals

- **Benefit:** Creates a manageable habit without forcing gamification.
- **Priority/Timing:** High Value / Later

- **Size/Risk:** Medium / Medium
- **Prerequisites:** Daily review queue and timezone decision
- **Data model:** Per-user daily target; completion derived from review events.
- **Backend:** Goal settings and daily completion query.
- **Frontend:** Goal setup and progress indicator.
- **Testing:** Timezone boundaries, target changes, partial completion.
- **Steps:** Define eligible activity → add preference → calculate progress → add UI.
- **Done:** Users can set a realistic target and see accurate daily progress.

# F15. Streaks

- **Benefit:** Encourages consistency for users motivated by continuity.
- **Priority/Timing:** Useful Later / Later
- **Size/Risk:** Medium / Medium
- **Prerequisites:** F14 and stable timezone rules
- **Data model:** Prefer deriving initially; cache only if necessary.
- **Backend:** Explicit qualifying-day and grace rules.
- **Frontend:** Modest display without punitive messaging.
- **Testing:** DST, travel/timezone changes, missed days, duplicate activity.
- **Steps:** Define semantics → implement pure calculation → expose → monitor behavior.
- **Done:** Streak calculations are stable, explainable, and not the product's primary incentive.

# F16. Word collections/tags

- **Benefit:** Organizes words by book, exam, topic, or personal goal.
- **Priority/Timing:** Useful Later / Later
- **Size/Risk:** Medium / Medium
- **Prerequisites:** One `UserWord` identity and mature filtering
- **Data model:** `Collection` and `CollectionWord` many-to-many relationship.
- **Backend:** Collection CRUD, membership, ownership, filtering.
- **Frontend:** Collection selector and collection page.
- **Testing:** Ownership, duplicate membership, archive/delete interaction.
- **Steps:** Validate user demand → add schema → CRUD → membership → filters.
- **Done:** A word can belong to multiple user-owned collections without duplicating learning state.

# F17. Import/export

- **Benefit:** Reduces lock-in and helps users migrate existing word lists.
- **Priority/Timing:** Useful Later / Later
- **Size/Risk:** Medium / Medium
- **Prerequisites:** Stable vocabulary schema and duplicate policy

- **Data model:** Usually none; optionally import-job records.
- **Backend:** Validated CSV/JSON export and previewable import.
- **Frontend:** Mapping/preview/errors/download flow.
- **Testing:** Malformed files, duplicates, encoding, large files, formula injection in CSV.
- **Steps:** Export first → define format → build dry-run import → confirm commit → report errors.
- **Done:** Users can safely export and import without silent data loss or duplication.

# F18. AI vocabulary explanations

- **Benefit:** Rephrases difficult definitions at the learner's level.
- **Priority/Timing:** Future / Experimental / Much Later
- **Size/Risk:** Medium / High
- **Prerequisites:** Stable core learning loop, AI policy, cost controls
- **Data model:** Optional cached generated content with model/version/source metadata.
- **Backend:** Provider abstraction, prompt templates, rate limits, moderation, disclaimer.
- **Frontend:** Explicit "Explain simply" action, not automatic replacement of canonical content.
- **Testing:** Evaluation set, unsafe prompts, cost, latency, hallucination review.
- **Steps:** Define narrow use case → prototype offline → evaluate → limited opt-in release.
- **Done:** Generated explanations are supplemental, attributable, bounded, and measurably useful.

# F19. AI-generated example sentences

- **Benefit:** Provides examples tailored to a selected meaning or proficiency level.
- **Priority/Timing:** Future / Experimental / Much Later
- **Size/Risk:** Medium / High
- **Prerequisites:** F4, AI controls, selected-meaning integrity
- **Data model:** Source, model, generation time, user acceptance/edit state.
- **Backend:** Meaning-grounded generation and validation.
- **Frontend:** Generate, accept, edit, or discard.
- **Testing:** Wrong sense, unsafe content, target-word absence, grammar quality.
- **Steps:** Build evaluation set → generate on demand → require user confirmation → never treat as canonical automatically.
- **Done:** Users control whether generated content becomes personal learning material.

# F20. AI vocabulary coach

- **Benefit:** Gives targeted explanations and practice suggestions based on current weak words.
- **Priority/Timing:** Future / Experimental / Much Later
- **Size/Risk:** Large / High

- **Prerequisites:** Review state, progress data, AI evaluation, privacy decisions
- **Data model:** Coaching sessions and bounded context references, not raw unlimited chat history.
- **Backend:** Tool-like access to selected vocabulary and review summaries.
- **Frontend:** Goal-oriented coaching workflow rather than a generic chatbot.
- **Testing:** Privacy, cross-user isolation, hallucinations, prompt injection, cost.
- **Steps:** Define 2–3 coaching jobs → prototype → evaluate against non-AI UX → limited release.
- **Done:** Coach recommendations are grounded in the user's data and improve a measured learning outcome.

## F21. Conversational vocabulary practice

- **Benefit:** Helps transfer learned vocabulary into active usage.
- **Priority/Timing:** Future / Experimental / Much Later
- **Size/Risk:** Large / High
- **Prerequisites:** F20, mature moderation, context quizzes, proven demand
- **Data model:** Practice session, target words, turns, feedback, retention policy.
- **Backend:** Constrained conversation orchestration and post-session evaluation.
- **Frontend:** Guided scenario, target-word prompts, feedback summary.
- **Testing:** Safety, target-word coverage, factuality, privacy, accessibility, cost.
- **Steps:** Start with scripted scenarios → add constrained AI turns → evaluate target usage → expand cautiously.
- **Done:** Practice reliably exercises chosen vocabulary rather than functioning as an unrestricted chat product.

---

# 8. 30-Day Roadmap

Assuming a small team and one-week work packages:

## Week 1

- Rotate/externalize JWT secrets.
- Secure/remove canonical write endpoint.
- Start password-hash migration.
- Confirm production-user and deployment assumptions.

## Week 2

- Complete adaptive password hashing.
- Add backend integration-test project.

- Cover login, registration, protected endpoints, and ownership.

## Week 3

- Fix transactional quiz counters and timestamps.
- Add duplicate-submission protection tests.
- Inventory duplicate `UserWord` records.
- Finalize one-word-per-user migration policy.

## Week 4

- Execute and validate `UserWord` relationship migration.
- Clean placeholder entity/DTO fields.
- Add archive/delete groundwork if capacity remains.

**30-day outcome:**
Known critical security problems are closed, learning statistics are reliable, one saved word has one identity, and critical backend behavior has automated protection.

---

# 9. 60-Day Roadmap

## Weeks 5–6

- Standardize API success/error contracts.
- Add global exception handling and validation.
- Introduce current-user accessor.
- Update Angular clients for new contracts.

## Week 7

- Extract external dictionary provider.
- Add provider timeout and failure handling.
- Handle concurrent cache misses.
- Correct unknown part-of-speech behavior.

## Week 8

- Extract personal vocabulary service.
- Add typed vocabulary query/update operations.
- Finish archive/delete, favorites filtering, and personal notes.

**60-day outcome:**
The backend has stable contracts and clean dictionary/vocabulary boundaries. Existing product capabilities are complete enough to support learning features safely.

---

# 10. 90-Day Roadmap

## Week 9

- Persist quiz sessions.
- Add restart, expiry, ownership, and idempotency tests.
- Add review-history APIs.

## Weeks 10–11

- Decompose `WordLookupComponent`.
- Add typed feature API services and authentication interceptor.
- Restore server-side paging, filtering, and debounced search.

## Week 12

- Replace the placeholder dashboard with a “Today” foundation.
- Add persistent navigation.
- Complete accessibility, dead-control, encoding, documentation, and test-runner cleanup.
- Design and migrate the initial learning-state model.

**90-day outcome:**
Vocabulary Builder is secure, maintainable, restart-safe, accessible, and ready to add a daily review queue without building on unstable foundations.

---

# 11. Longer-Term Product Roadmap

## Months 4–5 — Core retention loop

- Mastery states
- Daily review queue
- Weak-word resurfacing
- Simple spaced repetition
- Review history
- Personal example sentences
- Basic progress dashboard

## Months 6–7 — Practice depth

- Typed recall

- Multiple quiz modes
- Usage/context questions
- Daily learning goals
- Carefully designed streaks

## Months 8–9 — Organization and portability

- Collections/tags if user demand supports them
- Export
- Previewable import
- More advanced progress views only when actionable

## Beyond Month 9 — Evaluated AI experiments

- Simple explanations
- Generated personal examples
- Narrow vocabulary coach
- Constrained conversational practice

AI work should proceed only when the non-AI daily learning loop demonstrates adoption and retention.

---

# 12. Top 10 Tasks in Exact Recommended Order

1. Rotate and externalize JWT signing secrets.
2. Secure or remove the public canonical word-write endpoint.
3. Replace SHA-256 password hashing with transparent legacy migration.
4. Establish backend integration tests for authentication and ownership.
5. Make quiz result persistence and `UserWord` updates transactional and idempotent.
6. Migrate to one `UserWord` per user and canonical word.
7. Remove misleading entity/DTO placeholder fields.
8. Standardize API contracts.
9. Add centralized exception handling, validation, and current-user access.
10. Separate external dictionary and personal vocabulary responsibilities from `WordService`.

---

# 13. Dependencies Between Major Tasks

```
Security fixes
├── JWT secret rotation
```

```
├── Password migration
└── Canonical-write authorization

Integration test foundation
├── Quiz correctness
│   └── Persisted quiz sessions
│       └── Review history
├── UserWord identity migration
│   └── Clean vocabulary service
│       ├── Notes/examples/archive
│       ├── Server-side filtering
│       └── Review state
│           ├── Daily review queue
│           ├── Weak-word review
│           ├── Spaced repetition
│           └── Progress dashboard
└── API contract standardization
    ├── Central error handling
    ├── Angular typed API clients
    └── WordLookupComponent decomposition

Stable learning loop
└── Advanced quiz modes
    └── Context practice
        └── Evaluated AI features
```

Key sequencing rules:

- Do not introduce review scheduling before quiz data is accurate.
- Do not migrate `UserWord` identity without duplicate-data analysis and integration tests.
- Do not substantially decompose Angular networking before the backend contract is stable.
- Do not build analytics before the underlying attempt and review data is reliable.
- Do not build AI coaching before the product has a functioning daily review loop.

---

# 14. Risks That Could Derail the Roadmap

## Unknown production data

Existing duplicate `UserWord` rows or legacy password hashes may complicate migrations.

**Mitigation:** Profile production-like data, take backups, rehearse migrations, and make cleanup idempotent.

## Product ambiguity around “saved word”

If the team has not agreed whether users learn words, parts of speech, or meanings, schema changes will continue to reverse direction.

**Mitigation:** Resolve the one-word/one-meaning decision before R5.

## Refactoring without characterization tests

Service and component extraction could quietly change response shapes or UX behavior.

**Mitigation:** Add integration and characterization tests before moving responsibilities.

## Frontend/backend contract drift

Independent changes could break login, vocabulary, and quiz flows.

**Mitigation:** Migrate one vertical slice at a time and keep shared API examples current.

## Scope expansion

Collections, analytics, gamification, and AI could displace the core review loop.

**Mitigation:** Require every feature to advance the stated learning loop and satisfy its prerequisites.

## Overengineering

Repositories, CQRS frameworks, microservices, NgRx, or distributed infrastructure could consume months without improving learning value.

**Mitigation:** Add abstractions only at demonstrated responsibility boundaries.

## Unclear scheduling rules

Spaced repetition can become difficult to explain and test.

**Mitigation:** Start with a simple deterministic algorithm and simulate it before deployment.

## Incomplete operational access

The team may not be able to rotate secrets or inspect the deployed database immediately.

**Mitigation:** Treat environment access as a formal prerequisite, not an informal follow-up.

## Test tooling instability

The Angular test suite currently does not terminate within the tested 120-second window.

**Mitigation:** Diagnose browser startup, asynchronous tests, open timers, and CI configuration before relying on the suite as a release gate.

---

# 15. What We Should Explicitly Not Work On Yet

- Microservices
- Kubernetes or distributed orchestration
- Generalized repository/unit-of-work abstractions over EF Core
- CQRS or mediator frameworks across every operation
- NgRx or another global frontend state framework
- A complete visual redesign before workflow and accessibility fixes
- Complex analytics dashboards before data correctness
- Highly configurable spaced-repetition algorithms
- Social sharing, leaderboards, or public profiles
- Classroom, teacher, or LMS capabilities
- Multiple dictionary providers without a demonstrated need
- Full multilingual architecture unless multilingual support becomes a committed requirement
- Generic chatbot functionality
- AI-generated canonical dictionary content
- AI-driven mastery decisions
- Conversational AI before the core review loop is adopted
- Native mobile applications before responsive web usage justifies them

---

# 16. Final Ordered Roadmap

| Order | Task | Type | Priority | Size | Dependency | Timing |
|---|---|---|---|---|---|---|
| 1 | Rotate and externalize JWT secrets | Fix | Critical | Small | None | Immediate |
| 2 | Secure/remove canonical word-write endpoint | Fix | Critical | Small | Admin decision | Immediate |

| | | | | | | |
|---|---|---|---|---|---|---|
| 3 | Replace SHA-256 with adaptive password hashing | Fix | Critical | Medium | Deployment/user inventory | Immediate |
| 4 | Add backend integration-test foundation | Fix | High | Large | Test database decision | Immediate |
| 5 | Correct quiz counters and transactional submission | Fix | Critical | Medium | Integration tests | Immediate |
| 6 | Migrate to one `UserWord` per user/word | Fix | Critical | Large | Data inventory, quiz correctness | Immediate |
| 7 | Clean entity and DTO placeholders | Fix | High | Medium | `UserWord` decision | Immediate |
| 8 | Standardize API contracts | Fix | High | Medium | Integration tests | Next |
| 9 | Centralize exceptions, validation, and user identity | Fix | High | Medium | API contract | Next |
| 10 | Extract external dictionary provider | Fix | High | Medium | Stable contracts | Next |
| 11 | Handle concurrent dictionary cache misses | Fix | High | Medium | Dictionary extraction | Next |
| 12 | Extract personal vocabulary service | Fix | High | Medium | `UserWord` migration | Next |
| 13 | Persist quiz sessions | Fix | High | Medium | Quiz correctness | Next |
| 14 | Add archive/delete behavior | Feature | Must Have | Small | Vocabulary service | Next |
| 15 | Add favorites filtering | Feature | Must Have | Small | Server query contract | Next |
| 16 | Add personal notes | Feature | Must Have | Medium | Clean vocabulary model | Next |
| 17 | Improve Angular API/auth structure | Fix | High | Medium | Stable API contracts | Next |
| 18 | Decompose `WordLookupComponent` | Fix | High | Large | Typed Angular APIs | Soon |

| | | | | | | |
|---|---|---|---|---|---|---|
| 19 | Restore server-side paging/search | Fix | Medium | Medium | Vocabulary service/API client | Soon |
| 20 | Fix accessibility, dead UI, tests, and documentation | Fix | Medium | Medium | Component restructuring | Soon |
| 21 | Add personal example sentences | Feature | High Value | Medium | Clean `SampleSentence` model | Soon |
| 22 | Add persisted review history | Feature | Must Have | Medium | Persisted quiz sessions | Soon |
| 23 | Add learning state and mastery levels | Feature | Must Have | Medium | Accurate review history | Soon |
| 24 | Add daily review queue | Feature | Must Have | Large | Learning state | Soon |
| 25 | Add weak-word review | Feature | Must Have | Medium | Learning state | Soon |
| 26 | Add simple spaced repetition | Feature | Must Have | Large | Review queue and state | Soon |
| 27 | Add progress dashboard | Feature | High Value | Medium | Reliable learning data | Soon |
| 28 | Add typed recall | Feature | High Value | Medium | Extensible quiz generation | Later |
| 29 | Add multiple quiz modes | Feature | High Value | Medium | Quiz boundary cleanup | Later |
| 30 | Add usage/context quizzes | Feature | High Value | Large | Quality example data | Later |
| 31 | Add daily goals | Feature | High Value | Medium | Daily review events | Later |
| 32 | Add streaks | Feature | Useful Later | Medium | Goals and timezone rules | Later |
| 33 | Add collections/tags | Feature | Useful Later | Medium | Stable `UserWord` identity | Later |
| 34 | Add export and previewable import | Feature | Useful Later | Medium | Stable vocabulary schema | Later |
| 35 | Pilot AI explanations | Feature | Experimental | Medium | Proven learning loop, AI controls | Much Later |

| | | | | | | |
|---|---|---|---|---|---|---|
| 36 | Pilot AI-generated examples | Feature | Experimental | Medium | Personal examples, AI controls | Much Later |
| 37 | Pilot a constrained AI coach | Feature | Experimental | Large | Reliable learning state | Much Later |
| 38 | Pilot conversational vocabulary practice | Feature | Experimental | Large | Coach evaluation and moderation | Much Later |